

Git Methodology and Team Policy

Luc and Friends
Wits Alternate Reality Game (WARG) Platform

August 2026

1 Introduction

This document outlines the version control and Git methodology policy for the development of the WARG Platform. It serves to standardize our collaborative workflow across the team.

2 Commit Guidelines

2.1 When to Commit: Atomic Commits

Commits must be atomic. An atomic piece of work represents a single, independent unit of development that does not rely on subsequent commits to function.

- A commit should focus on one specific task
- This approach ensures that if a change needs to be reverted, only the isolated feature or fix is rolled back without affecting unrelated code.

2.2 How to Name Commits

We adhere strictly to the Conventional Commits standard. Commits should be structured as `<type>[optional scope]: <description>`.

We use the following specific types, and every commit must fall into one of these categories with an appropriate description:

- `feat`: Adding entirely new functionality.
Example: `feat(auth): add OAuth login`
- `fix`: Resolving an unintended behavior or bug.
Example: `fix(map): correct marker alignment`
- `chore`: Routine maintenance, dependency updates, or build script changes.
Example: `chore: update npm packages`
- `test`: Adding or refactoring automated tests.
Example: `test(api): add unit tests for user validation`

- docs: Writing or updating documentation.
Example: docs: update README installation steps
- ci: Changes to CI/CD pipelines.
Example: ci: add GitHub Actions linting
- revert: Reverting a previous commit that caused issues.
Example: revert: feature-payment-gateway
- hotfix: Urgent, critical fixes applied directly to bypass standard staging.
Example: hotfix: patch security vulnerability in auth
- perf: Code changes specifically improving performance.
Example: perf: implement image lazy loading
- refactor: Restructuring code without adding features or fixing bugs.
Example: refactor: extract header component

3 Branching Strategy

3.1 When to Branch (GitHub Flow)

Following the GitHub Flow methodology, the `main` branch must always be in a deployable state. You must create a new branch off `main` whenever you begin work on any new wide-scoped feature, bug fix, or routine chore. Development should never occur directly on the `main` branch except for trivial hotfixes.

3.2 How to Name Branches

Branch names should clearly indicate the type of work and use a descriptive hyphenated name. The prefixes directly mirror our commit types:

- feat/: e.g., feat/dark-mode-toggle
- fix/: e.g., fix/checkout-button-alignment
- chore/: e.g., chore/update-react-router
- test/: e.g., test/cypress-login-flow
- docs/: e.g., docs/update-installation-guide
- ci/: e.g., ci/add-linting-step
- revert/: e.g., revert/feature-payment-gateway
- hotfix/: e.g., hotfix/stripe-payment-failure
- perf/: e.g., perf/image-lazy-loading
- refactor/: e.g., refactor/extract-header-component

4 Merging and Pull Requests

4.1 When to Merge (GitHub Flow)

Under GitHub Flow, once a Pull Request has been opened, reviewed, and all automated checks pass, it is ready to be merged. The code is merged directly into the `main` branch.

4.2 Requirements for Merging

Before any branch is merged, the following questions must be satisfied:

- Peer Review: Has someone else reviewed the code? Does it do what it claims to do?
- Automated Testing: Does the code pass all automated testing frameworks established in our CI/CD pipeline?
- Deployment Verification: Does the code successfully compile and deploy?

Merging random, unreviewed code is prohibited and punishable.

4.3 Review Rotation Policy

To distribute the review workload among the team, we operate on a rotating review schedule (rotated per sprint or week depending on present workloads).

- One member is assigned to review all standard Pull Requests.
- A second member is assigned specifically to review the primary reviewer's PRs, ensuring no code is merged without oversight.

5 Versioning

We will utilize Semantic Versioning (`Major.Minor.Patch`) for our project releases.

- Major: Fundamental changes to the application, such as API overhauls.
- Minor: Backward compatible changes and additions.
- Patch: Bug fixes, refactors, and minor adjustments.

AI Declaration: The preceding document was partially generated, reviewed and edited with: Gemini Web[Gemini 3.1 Pro]